

ECE 457B – Assignment 2

Will McIntyre (21009235)

October 6th, 2026

Question 1.1

Squared Loss $\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}\left(t^{(i)} - y^{(i)}\right)^2$ can be used for both regression and classification. It is the standard loss function for predicting continuous variables but can also be applied to classification by treating discrete class labels (such as 0 and 1) as numeric target values. There is an assumption that the model outputs continuous real numbers. An advantage is that the function is smooth and differential everywhere, meaning it is suitable for gradient descent. A disadvantage is that it squares errors which means large errors are heavily penalized and can dominate training.

Question 1.2

Mean Absolute Error $\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N} | t^{(i)} - y^{(i)}|$ is used for regression primarily but can be used in classification similarly to squared loss by treating discrete class labels as target values in linear classification. The output is assumed to be a continuous real number. An advantage of the mean absolute error is that since it is linear, it is less prone to outliers skewing the training. However, compared with MSE, small errors get penalized more and is also not differentiable at zero residual, making it unsuitable for gradient descent.

Question 1.3

Binary Cross Entropy $\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\left[t^{(i)}\log\left(y^{(i)}\right) + \left(1 - t^{(i)}\right)\log\left(1 - y^{(i)}\right)\right]$ is used in binary classification. The output is assumed to represent probabilities bounded between 0 and 1, and the true target labels are values of 0 or 1 following a Bernoulli distribution. An advantage for this loss function is that it heavily penalizes highly confident but incorrect predictions. A disadvantage to Binary Cross Entropy is that it is vulnerable to class imbalance. BCE treats every data point equally, and if one class makes up 99% of your data, a model can achieve very low loss by simply guessing that majority class each time.

Question 1.4

The Huber Loss Error is used in regression, but again like MAE and MSE could hypothetically be used in linear classification as well. The model output is assumed to be continuous real numbers. An advantage of this loss function is that it provides a balance between squared and absolute loss that offers robustness against outliers while remaining differentiable near the minimum. A disadvantage is that it introduces an additional hyperparameter δ that must be tuned.

Question 2.1

To randomly split the dataset into 80% training, 10% validation and 10% testing, two Python libraries were used. Pandas was used to read the csv and convert the data to a Dataframe object, which was then passed into sklearn's train_test_split function that accepts a test size for the split percentage. I split the data into 80% training to begin and then split the remaining 20% into 10% validation and 10% testing using this function. I made this split reproducible by passing the random_state argument into both function calls with a value of 123123. This hardcodes a seed for the random number generator, guaranteeing that the same rows will be assigned to the desired sets every time the script is executed.

Question 2.2

Using the split from 2.1 with sklearn's default models for KNN, Decision Tree, Linear Regression (with thresholding at midpoint 0.5) and Logistic Regression Classifier led to the following validation accuracy readings:

Table 1: Validation Set Accuracy Across Different Models

Model	Validation Accuracy (%)
KNN	99.3
Decision Tree	98.8
Linear Regression	98.8
Logistic Regression	99.2

These validation accuracy readings are near perfect for each model, ranging from 98.8% to 99.3% with KNN reporting the highest accuracy. These scores lead me to believe that there could be heavily class imbalanced data.

Question 2.3

Overall accuracy is problematic because it hides a model's failure to identify minority classes in imbalanced datasets. Because accuracy evaluates performance on the entire dataset, a model can achieve near-perfect scores like those seen above by just predicting the majority class for most instances. This is a problem in situations where identifying rare minority classes is the actual objective.

Question 2.4

Exploring the dataset's labels using pandas led to the following counts:

Table 2: Dataset Class Makeup

Class	Count
0	9900
1	100

This analysis shows that the dataset has two total classes and there is a severe imbalance, with class 0 taking up 99% of the total data. Figure 1 below shows a scatterplot of the data based on inputs x_1 and x_2. The plot clearly shows this imbalance visually, with far more red dots indicating a target class of 0 dominates the dataset.

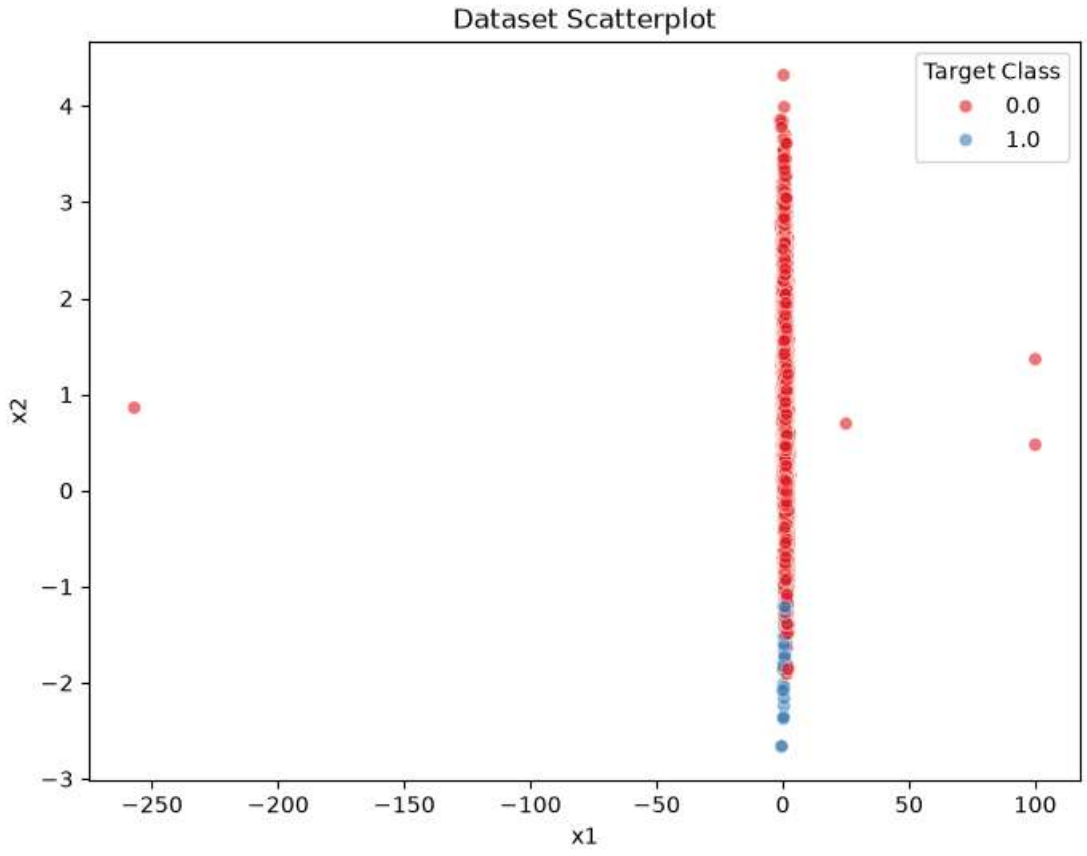


Figure 1: Dataset Scatterplot

Question 2.5

Using sklearn's confusion_matrix function, confusion matrices were calculated for each model, shown in Table 3. The confusion matrix standard form output from this function is shown below as well, where positive represents data classified as class-1 and negative represents class-0.

TN	FP
FN	TP

Table 3: Model Confusion Matrices and Classification Performance

Model	Confusion Matrix		True Class-0 Classified (%)	True Class-1 Classified (%)
KNN	986	2	99.8	58.33
	5	7		
Decision Tree	981	7	99.29	58.33
	5	7		
Linear Regression	988	0	100	0
	12	0		
Logistic Regression	987	1	99.9	41.67
	7	5		

The overall validation accuracy of all these models was in the range of 98.8% to 92.2%. However, when we look at the per-class classification performance for these models, we see very different results. The models all have very poor performance in classifying class-1 targets, ranging from 0% in linear regression to 58.33% correctly classified in KNN/Decision Tree. The models achieved their high accuracies because of the imbalance and lack of minority class (class-1) datapoints and this shows that overall accuracy can be a misleading metric for measuring model performance.

Question 2.6

Precision and recall are useful in the presence of data imbalances in classes because of their focus on minority positive class (class-1). Both precision and recall do not include true negatives, which dominates the dataset. With accuracy, the true negatives are in the numerator which can cause high accuracy readings if the negative class makes up most of the dataset and the model predicts it consistently. Precision and recall evaluate the models based on its ability to handle the positive class which better shows performance in imbalanced cases because it ignores the dominant true negatives. This answer assumed that you have mapped minority classes to the positive label.

Question 2.7

To calculate the F_1 score for positive classes that combine precision and recall through their harmonic mean, sklearn's f1_score function was used for each model. To handle an undefined precision from zero-division, the zero_division argument to the function was set to zero. This meant that if both true positives and false positives equal zero (such as models that default predict class-0 for every validation example like Linear Regression), the precision would be zero instead of undefined due to a division by zero. Table 4 displays the F_1 scores for the models, showing poor performance when classifying class-1 targets.

Table 4:F_1 Scores for Models Predicting Class-1

Model	**F_1 Score**
KNN	0.6667
Decision Tree	0.5385
Linear Regression	0
Logistic Regression	0.5556

Question 2.8

Using sklearn to randomly duplicate the minority class (class-1) instances in the training set and then re-training the models and comparing against the same validation set led to the following results:

TN	FP
FN	TP

Table 5: Over-Sampled Training Data Confusion Matrices and Performance for Positive (class-1) Labels

Model	Confusion Matrix		Accuracy (%)	Precision (%)	Recall (%)
KNN	975	13	98.3	38.1	66.67
	4	8			
Decision Tree	983	5	99.0	58.33	58.33
	5	7			
Linear Regression	877	111	88.9	9.76	100
	0	12			
Logistic Regression	901	87	91.3	12.12	100
	0	12			

Selecting the best model depends heavily on the cost of false positives versus false negatives. For example, if the model is for something like cancer detection, we would maybe want to prioritize recall to catch all positives. However, without specific context the decision tree model is the best performer overall. While linear regression and logistic regression achieved perfect recall by finding every true class-1 example, they had many false positives leading to poor precision and lower overall accuracy when compared to the other models. The decision tree maintains the highest precision with only 5 false positives, while still catching over half of the minority class (recall of 58.33%).

Because the models were trained on a dataset where class-1 appeared 50% of the time, they became far more eager to predict class-1 on the validation set. This led to spikes in false positives, but improved recall as the models more often predicted all the positives as true positives. In practice, we should treat the over-sampling amount as a tunable hyperparameter. By training with multiple different majority to minority ratios, we could select the oversampling amount that maximizes a metric like the F_1 score that contains both precision and recall metrics.

Question 2.9

Using sklearn to under sample the majority class (class-0) to match the minority-class amount led to the following results:

TN	FP
FN	TP

Table 6: Under-Sampled Training Data Confusion Matrices and Performance for Positive (class-1) Labels

Model	Confusion Matrix		Accuracy (%)	Precision (%)	Recall (%)
KNN	883	105	89.5	10.26	100
	0	12			
Decision Tree	907	81	91.8	11.96	91.67
	1	11			
Linear Regression	878	110	88.9	9.76	100
	0	12			
Logistic Regression	908	80	92	13.04	100
	0	12			

Table 7: Original Training Data Confusion Matrices and Performance for Positive (class-1) Labels

Model	Confusion Matrix		Accuracy (%)	Precision (%)	Recall (%)
KNN	986	2	99.3	77.78	58.33
	5	7			
Decision Tree	981	7	98.8	50	58.33
	5	7			
Linear Regression	988	0	98.8	0	0
	12	0			
Logistic Regression	987	1	99.2	83.33	41.67
	7	5			

Compared to the original set, shown in Table 7, which had high accuracy but not great recall, under sampling drastically improves recall because the model is no longer biased toward the majority class. However, compared to the over-sampled set, shown back in Table 5, which provided a moderate balance of precision and recall, under-sampling caused precision to crash entirely to the 9-13% range across all models. The general trade-off is sacrificing the reliability of positive predictions (precision) for catching every positive case (recall). We also surrender the comprehensive, true distribution of the majority class for class balance by shrinking its size.

The crash in precision is directly tied to the discarded data caused by forcing a 1:1 ratio of majority to minority class training data. By under sampling, we threw away over 98% of the majority class

(class-0) training data which led to the models losing critical information about that majority class. This loss of information led to all the models predicting many false positives.

As stated about oversampling, the retention ratio could be treated as a tunable hyperparameter. You could create different training sets with different amounts of the majority class and then measure performance again the validation set by minimizing the F_1 score.

Question 2.10

Exploring the dataset using a boxplot for input features x_1 and x_2 led to the plot shown below in Figure 2. The solid central plot represents the middle 50% of data, and the bottom/top edges represent the 25th and 75th percentile respectively.

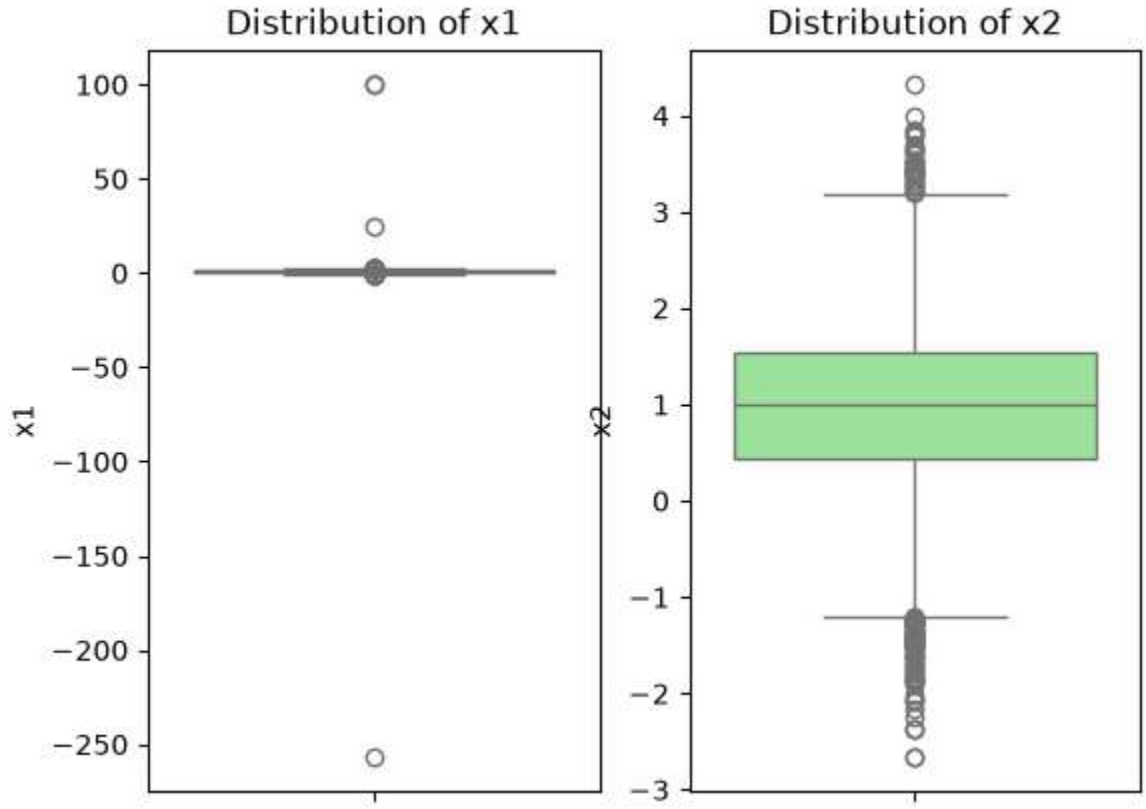


Figure 2: Boxplot Distribution of Input Features

The plot shows that feature x_1 exhibits extreme outliers located at approximately -250, 25 and 100, with all remaining points around 0. Feature x_2 displays a dense cluster of moderate outliers extending above and below its interquartile range, but not at the significance as those in feature x_1.

To deal with these features, it is important to look at the targets corresponding to the outliers. If we look at the dataset scatterplot with class labels found earlier and shown in Figure 3, we can see that the extreme outliers in the x_1 feature are all part of class-0. The moderate outliers for the x_2 feature in the lower values seem to be where the class-1 targets reside. Based on this analysis, we would want to eliminate the extreme outliers in the x_1 feature set, while preserving the x_2 outliers to avoid erasing our minority class. First, we could completely remove the major outliers from the dataset because we visually confirmed they were a part of the same target class. Alternatively, we could set a statistical floor/ceiling on the x_1 feature (like the 1st and 99th percentile) and then overwrite any value that exceeds these limits to match the boundary. This allows us to keep the datapoint's x_2 feature value while eliminating the significant outliers in x_1.

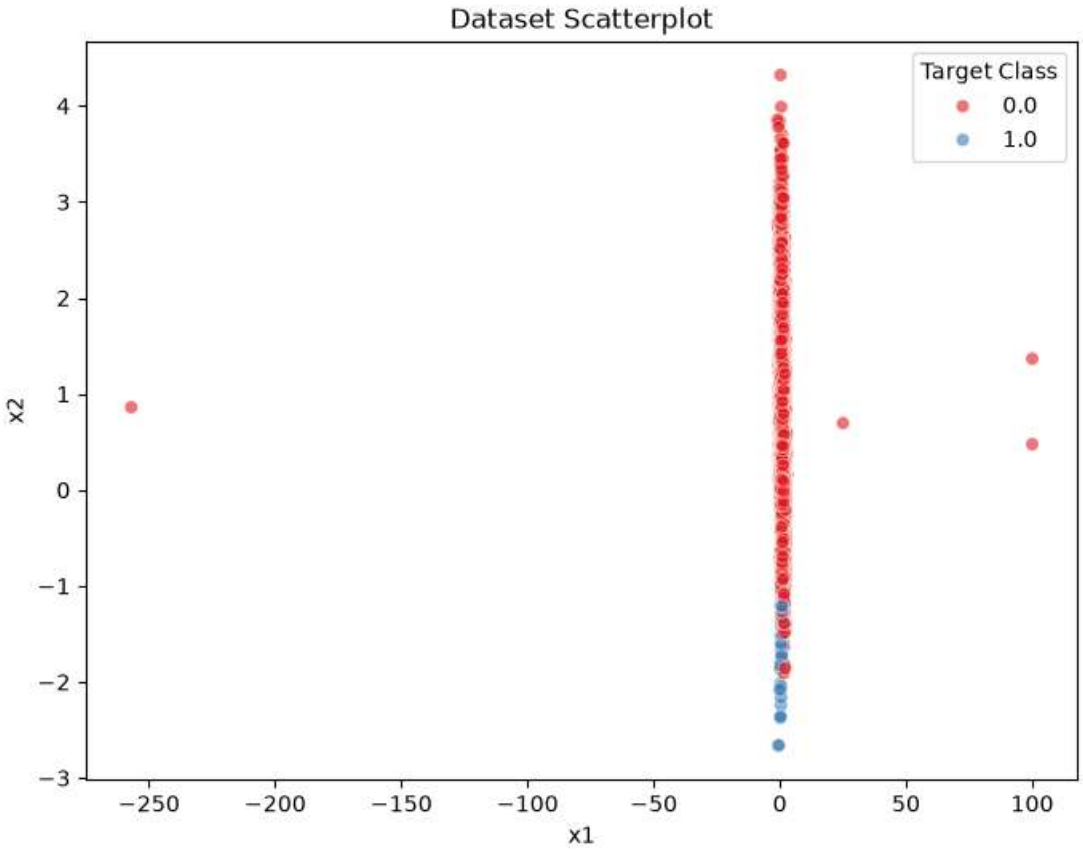


Figure 3: Dataset Scatterplot with Target Class

Question 2.11

Using the original unsampled training set, three KNN models were trained with $k \in \{1, 3, 5\}$ and evaluated based on their validation confusion matrices, shown in Table 8.

TN	FP
FN	TP

Table 8: Validation Confusion Matrices with Varying Number of Neighbours

Number of Neighbours	Confusion Matrix	
1	984	4
	5	7
3	985	3
	5	7
5	986	2
	5	7

Looking at the confusion matrices, we can see that when k is sufficiently large, a single noisy outlier will be outvoted by the of normal data points. We can see this as k increased from 1 to 5, the false positives went from 4 to 2 because the model stopped creating boundaries around noise. Outliers are a problem when $k = 1$ and the model memorizes the training data. If a validation point lands close to an outlier, then the model will blindly copy the outlier's class, causing an overfitted decision boundary.

Question 2.12

Using the original unsampled training set, three decision tree models were trained with max depths of $\{1,2,3\}$ and evaluated based on their validation confusion matrices, shown in

TN	FP
FN	TP

Table 9: Validation Confusion Matrices with Varying Max Depths in Decision Tree

Max Depth	Confusion Matrix	
1	984	4
	5	7
2	988	0
	5	7
3	988	0
	5	7

Decision trees are generally robust to outliers with extreme magnitudes because instead of calculating distances between points, they produce threshold-based splits. If the outlier lands on correct side of the split, the tree treats it the same way. However, outliers can become destructive if the decision tree is allowed to grow too deep. Like a low number of neighbours in the KNN case, a max depth that is set very high can overfit the training data by creating hyper-specific branches for noisy outliers.

Question 2.13

Neither decision trees of KNN models can be considered uniformly more robust to outliers. Their vulnerability depends on how their hyperparameters are tuned and the data itself. KNN is highly sensitive to outliers when k is small which creates an overfitted decision boundary. Decision trees are highly sensitive to outliers when max depth is left unrestricted, which leads to overfitting as well. In terms of data properties, if features have noticeable outliers with large magnitudes, then they will dominate the distance formula used in KNN. Trees on the other hand do not care about the numerical magnitude between data points and instead splits based on a number. For example, whether the x_1 values are $[1, 2, 300]$ or $[1, 2, 3]$, a split drawn between the second and third items separates the data in both scenarios. This makes them more robust to large magnitude outliers than KNN and can lead to more favourable behaviour.

Question 2.14

There are a variety of potential issues with real-world data that quire attention before training. To begin, real world datasets can contain incomplete records or empty readings due to data collection errors or sensor failures. This matters because our machine learning algorithms require complete sets of input features mapped to targets. These incomplete datapoints would need to be filtered out prior to training. Another potential issue with real-world data is inherent data bias. Real world collection capture historical and social prejudices. If the training set misses specific groups or edge cases then the model makes unfair, skewed or inaccurate predictions. For example, a model that is trained only on winter weather data would not be suitable outside of that season, or a resume screener could penalize certain candidates based on unfair traits. This was shown in 2017 when Amazon scrapped an AI recruiting tool that discriminated against women as the model was trained on employment datasets historically dominated by men[1].

[1] https://www.reuters.com/article/world/insight-amazon-scraps-secret-ai-recruiting-tool-that-showed-bias-against-women-idUSKCN1MK0AG/

Question 3.1

Using sklearn for the base model using default hyperparameters and training 100 decision trees led to the following histogram of the base classifiers and overall ensemble accuracy.

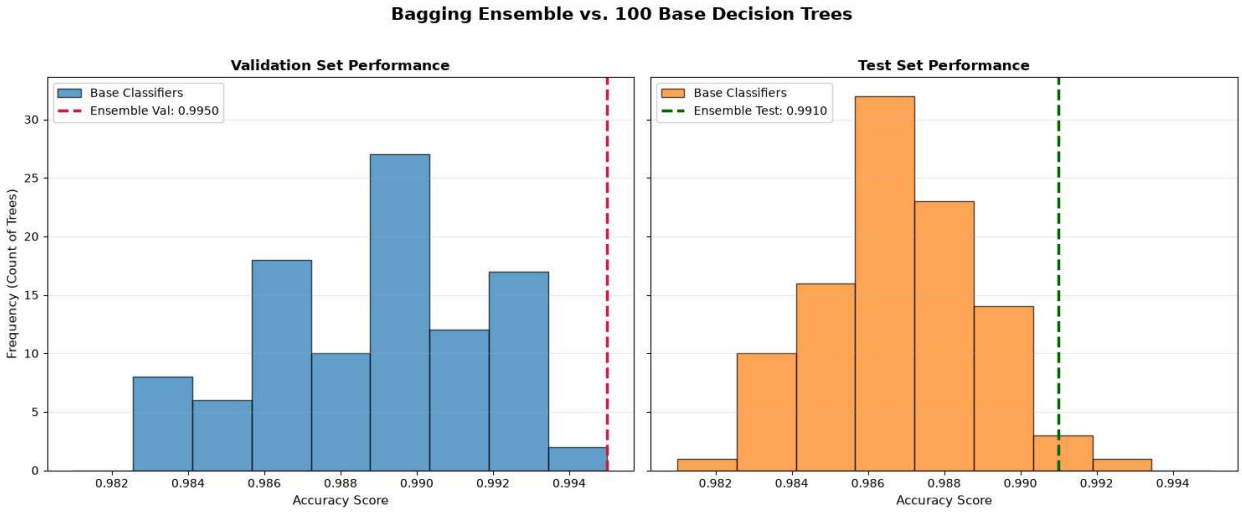


Figure 4: Histograms of Bagging Ensemble Performance vs Base Decision Trees in Test and Validation sets

We can see that the ensemble classifier has very good accuracy compared to most base classifiers. Bagging averages out the extreme fluctuations and variance of individual trees, producing a more accurate model.

Question 3.2

Table 10 displays a variety of different hyperparameter adjustments in the base decision trees and their effects on average base tree accuracy and ensemble validation/test accuracy.

Table 10: Base Model Hyperparameter Adjustments and Ensemble Accuracy

Hyperparameter	**Avg Base Tree Accuracy (Validation)**	**Ensemble Accuracy (Validation)**	**Ensemble Accuracy (Test)**
max_depth=None	98.89%	99.5%	99.1%
max_depth=1	99.07%	99.1%	98.5%
max_depth=2	99.43%	99.5%	99.2%
max_depth=5	99.35%	99.5%	99.2%
min_samples_split=20	99.03%	99.5%	99.2%
min_samples_leaf=10	99.32%	99.5%	99.2%
criterion='entropy'	98.91%	99.5%	99.2%

Restricting trees to lower depth resulted in the lowest ensemble test and validation set accuracy. Once we increased the max depth to 2, the ensemble accuracy reached its peak performance of 99.2%. Constraining split thresholds through min_samples_split and min_samples_leaf produced identical ensemble performance while slightly stabilizing the individual base classifiers. Neither the entropy criterion nor setting max features to 1 produces ensemble accuracy changes compared to no max depth. One trade-off to consider is that allowing decision trees to grow larger (by increasing max depth) can improve the ensemble accuracy but will increase the computational costs of coming up with a prediction.

Question 3.3

To average the predictions of the base classifier, where N = 100 trees and $\hat{y}_n$ is the predicted class:

$$\bar{y} = \frac{1}{N}\sum_{b=1}^{B} \hat{y}_n$$

We can then assign a final discrete class from this average using a threshold of 0.5. The class predictions do not change as requiring $\bar{y} > 0.5$ is mathematically identical to requiring more than 50 of 100 trees voting for class 1, which is the same as the majority vote.

Question 3.4

There are a variety of benefits and drawbacks of using an ensemble method. First, a benefit is variance reduction. Individual decision trees can have high variance and are sensitive to small changes in the training data. Averaging over many samples significantly reduces this variance. There is also improved generalization and thus smoother decision boundaries as ensemble models mitigate overfitting to outliers or noise in the training set. One drawback is interpretability. A single decision tree can be followed to find exactly how the output was achieved, but an ensemble of 100 of these trees makes it far harder to tell. Additionally, there is far higher computational overhead from training more decision trees.

An ensemble could underperform a single classifier under certain conditions. For example, if the base model is very constrained (for example depth 1), bagging cannot fix the inherent high bias. A single deeper tree could perform better than a bagged ensemble of underfit trees. In our case, based on experimental results, the ensemble does not underperform the base classifier as seen in the previous section. The average base tree accuracy is lower than the ensemble accuracy in the validation set for all hyperparameter configurations.

Question 3.5

In our simple bagging model, we only sampled the training data points with replacement. At every decision node in every tree, the algorithm still considers all available features to find the best split. In random forest classification, there is still the bagging on training data, but there is also random feature subsampling at every split [2]. When deciding where to split a node, the algorithm is forced to choose from a random subset of features. This is because in simple bagging if some features are very strong predictors for the target, then these features will be selected in many of the trees, leading to correlation and limiting variance reduction. The random feature selection eliminates this potential for correlation between the base trees.

[2] https://en.wikipedia.org/wiki/Random_forest

Question 3.6

Adaptive boosting (AdaBoost) assigns weights to training examples and updates these weights based on prior model results (specifically to emphasise misclassified examples). Using the same 80% training set as before, AdaBoostClassifer was used to build three different models with different base estimators. Each model's validation set accuracy is shown below:

Table 11: AdaBoost Classifier Accuracy with Different Base Estimators

Model	**Validation Set Accuracy**
Default AdaBoost	99.2%
Decision Tree AdaBoost	98.8%
Random Forest AdaBoost	99.5%

The default AdaBoost estimator, set to None uses a depth-1 decision tree as the base estimator. Because a tree of single depth is highly constrained and cannot overfit the dataset on its own, it has some misclassified points. This allows the sequential form of the AdaBoost algorithm to work as it increases the weight of these misclassified points which forces the next tree to focus on them. This approach combines many weak classifiers into a high generalized model with 99.2% validation set accuracy.

On the other hand, using a decision tree with unlimited depth decreases the overall validation set accuracy to 98.8%. A standard decision tree acts as a strong learner that grows until it memorizes the training data which can include noise. And because the first tree in the sequence makes zero errors on the training set, AdaBoost has no misclassified points to re-weight. So, the ensemble degrades to what is effectively a single overfitted decision tree.

Finally, using a Random Forest embeds a bagging ensemble inside a boosting ensemble. While this brute-force approach achieved the best validation accuracy, it would achieve diminishing returns. Random Forests are already highly optimized strong learners designed to reduce variance, as discussed earlier. Feeding them into AdaBoost leaves almost no room for iterative bias reduction, making the boosting strategy just more overhead on top of an already computationally intensive bagging ensemble that is Random Forest.

Question 3.7

Both AdaBoost and Gradient Boosting are sequential boosting algorithms that convert weak learners into a strong ensemble. Their difference is how they identify and correct errors. AdaBoost reweights (adjusts the importance) of the individual data points. It identifies misclassified examples after each round and increases their sample weights while decreasing correctly classified points [3]. Subsequent learners are trained on the reweighted dataset to force it to learn through difficult examples. On the other hand, in Gradient Boosting the data weights never change, but instead it changes the target labels that the next tree tries to predict. It evaluates the ensemble's current predictions against the true labels and computes the errors (negative gradient of loss function) [4]. The next tree is trained to predict the errors directly instead of predicting the target.

[3] https://en.wikipedia.org/wiki/AdaBoost
[4] https://en.wikipedia.org/wiki/Gradient_boosting

Question 3.8

Looking at validation accuracy, test accuracy and training time for the AdaBoost, Gradient Boosting and XGBoost led to the following results under default implementations:

Table 12: Classification Methods and Accuracy/Training Time

Classifier	**Validation Accuracy**	**Test Accuracy**	**Training Time (s)**
AdaBoost	99.2%	98.6%	0.213
Gradient Boosting	99.3%	99.0%	0.7149
XGBoost	99.0%	99.0%	0.3388

All three ensemble methods achieved great overall accuracy around 99%. While standard Gradient Boosting achieved the highest validation accuracy, both Gradient Boosting and XGBoost achieved the same accuracy on the test set which was slightly higher than AdaBoost. This indicates that gradient based methods generalize slightly better to unseen data than AdaBoost. The core advantage of XGBoost over Gradient Boosting is its training speed. Standard Gradient Boosting was the slowest model and XGBoost cut this training time by more than half. The parallelization of mathematical sorting helps reduce this execution time while achieving similar results to regular Gradient Boosting.